

IT360 – Information Assurance and Security Course

StegoCipher

Image Steganography System

Deliverables – Week 3 & 4

Part 3 – High Level Design (HLD) Document
Part 4 – Tools, Tech Stack & Development Phases

Submitted by: Sarah Mneja, Maryem Ammar, Ayoub El Atri
Academic Year: 2025–2026
Professor: Manel Abdelkader

Part 3 – High Level Design (HLD) Document

3.1 Purpose of the HLD

This High Level Design document describes the architecture, components, interfaces, and data flows of StegoCipher — a client-side web application that hides secret text messages inside ordinary image files using the Least Significant Bit (LSB) steganography technique. It serves as the contractual blueprint guiding all development, testing, and code review activities for Project 6 of the IT360 Cybersecurity Course.

3.2 System Scope & Actors

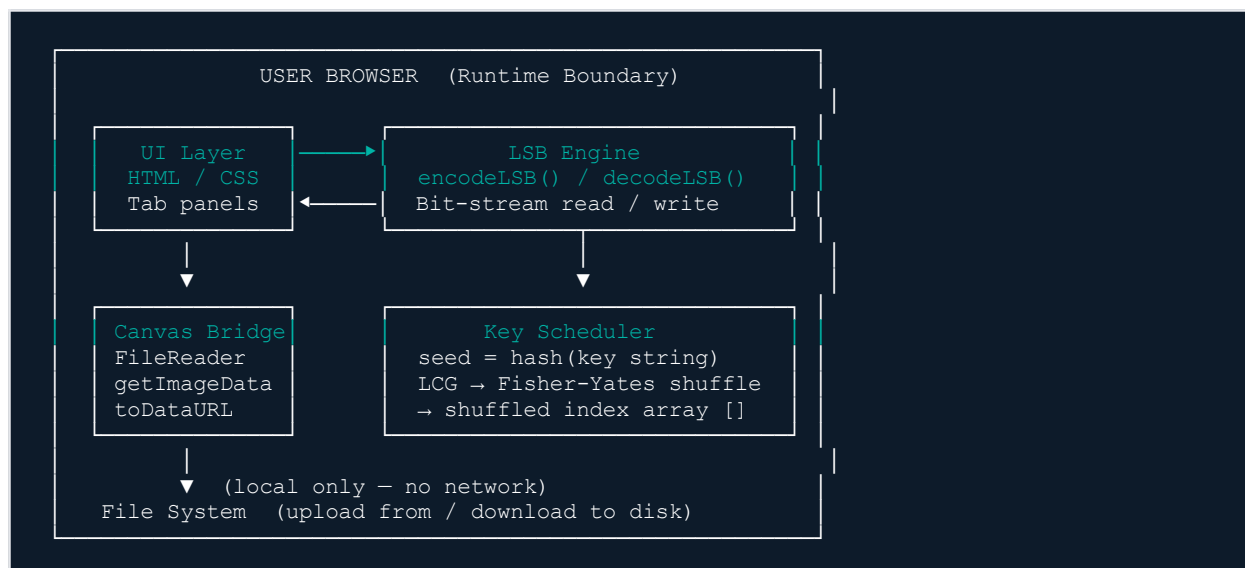
The system is a single-page web application that allows two categories of users to perform complementary steganographic operations. Because the application is a pure client-side SPA with no backend server, all operations run entirely in the browser. No image data or messages are ever transmitted to a server or stored persistently.

Actor	Role	Interaction with system
Sender	Embeds a secret message inside a cover image and downloads the stego image.	Uploads cover image (or uses default carrier), types secret message, optionally sets a secret key, downloads output PNG.
Receiver	Extracts the hidden message from a received stego image.	Uploads stego PNG, enters same key used during encoding, reads recovered plaintext in the output panel.

3.3 Architecture Overview

The system adopts a single-tier, client-side-only architecture. The complete application is delivered as a single self-contained HTML file. All computation occurs locally in the browser. There is no build step, no framework, no npm, and no server.

Architectural layer	Description
Presentation layer	A single HTML file containing all markup, embedded CSS (dark theme, CSS variables, animations), and embedded JavaScript. No external framework — pure vanilla JS.
Processing layer	JavaScript functions executing in the browser: the LSB Engine, the Key Scheduler, and the Canvas Bridge. These access the HTML5 Canvas API to read and write pixel data via ImageData.
Storage layer	None (stateless). The browser File API reads the uploaded cover/stego image into memory. The stego image is generated in memory and made available for download via <code>canvas.toDataURL("image/png")</code> . No <code>localStorage</code> , no cookies, no server-side storage.
Hosting layer	None required. The file is distributed as a standalone download. It can optionally be hosted on Netlify, or any static host — but it also runs correctly when opened directly from the filesystem (no SubtleCrypto dependency).



3.4 Component Breakdown

The application is structured into four logical components, all co-located inside the single HTML file. The Metrics Module and separate File Handler class from the original design have been merged into the UI Layer and Canvas Bridge respectively.

Component	Responsibility
UI Layer	Manages the DOM, handles user events (file uploads via input and drag-and-drop, button clicks, tab switching between Encode and Decode modes). Orchestrates calls to the LSB Engine and Canvas Bridge. Displays progress bars, status messages, capacity bar, and result metadata. The only component that directly touches the DOM.
Canvas Bridge	Bridges the browser File API and HTML5 Canvas API. Converts an uploaded image file into an ImageData pixel array for the LSB Engine, and converts a modified pixel array back into a downloadable PNG. Also generates the default carrier image (canvas gradient + noise) on page load.
LSB Engine	The core algorithm. Converts a UTF-8 message string to a bit array (prepended with a 32-bit big-endian length header). Writes each bit into the LSB of a pixel channel (R, G, or B — alpha is never touched). For decode, reads bits from the same channel sequence and reconstructs the UTF-8 string via TextDecoder.
Key Scheduler	Derives a 32-bit integer seed from the user key string via polynomial rolling hash ($\text{seed} = \text{seed} \times 31 + \text{charCode}, \text{mod } 2^{32}$). Feeds that seed into a Linear Congruential Generator (multiplier 1664525, increment 1013904223). Uses the LCG to perform a Fisher-Yates shuffle over the full list of available pixel channel indices. Without the correct key, the bit sequence cannot be reconstructed.

3.4.1 UI Layer

The UI Layer renders two tab panels — Encode and Decode — using CSS-driven tab switching. It listens for:

- The image file input change event and drag-drop events on the upload zones → calls Canvas Bridge to load the image and compute capacity.
- The secret-text textarea input event → updates the live capacity bar (byte count and colour coding: green < 70%, yellow < 90%, red ≥ 90%).
- The Encode button click → validates inputs, calls `encodeLSB()`, draws the result to the result canvas, populates metadata chips, shows download button.
- The Decode button click → validates that an image is loaded, calls `decodeLSB()`, displays the extracted string.
- The Download button click → calls `canvas.toDataURL("image/png")` and triggers a programmatic anchor click.

3.4.2 Canvas Bridge

On image upload, the Canvas Bridge draws the image onto an off-screen work canvas (`id="work-canvas"`) and calls `getImageData()` to extract the full RGBA pixel array. On page load it generates a 512×512 gradient+noise default carrier so the user can test immediately without uploading an image.

After encoding, it draws the modified `ImageData` back onto the result canvas (`id="result-canvas"`) for preview. The download uses `canvas.toDataURL("image/png")` — always PNG, never JPEG, to preserve LSB data.

3.4.3 LSB Engine

Encoding steps:

- `TextEncoder.encode(message)` → `Uint8Array` of UTF-8 bytes.
- Build a 32-bit big-endian length header (4 bytes) and prepend to the message bytes.
- Unpack all bytes into a flat bit array (MSB first per byte).
- Iterate over bits; for each bit, take the next channel index from the shuffled list, set `pixels[idx] = (pixels[idx] & 0xFE) | bit`.
- Call `ctx.putImageData(imgData, 0, 0)` to write the modified pixel array back to the canvas.

Decoding steps:

- Re-generate the same shuffled index list from the key.
- Read 32 bits from the first 32 shuffled channels → reconstruct the 32-bit length `N`.
- Read `N × 8` bits from the next `N × 8` shuffled channels → `Uint8Array` of `N` bytes.
- `TextDecoder("utf-8", { fatal: true }).decode(bytes)` → UTF-8 string. If decoding fails, the wrong key was used.

3.4.4 Key Scheduler

`getShuffledIndices(count, key):`

- If key is empty, return the identity array `[0, 1, 2, ... count-1]` (sequential, no shuffle).
- Hash key string: `seed = 0`; for each char: `seed = (seed × 31 + charCode) >>> 0`.
- LCG: function `lcg()` { `seed = (seed × 1664525 + 1013904223) >>> 0`; return `seed / 0x100000000`; }
- Fisher-Yates: for `i` from `count-1` down to 1, swap `indices[i]` with `indices[floor(lcg() × (i+1))]`.
- Return the shuffled array.

3.5 Data Flows & Interface Contracts

All interfaces are in-browser JavaScript function calls — there are no network API contracts

3.5.1 Encode Flow

Step	From	To	Data passed
1	User	UI Layer	File object (image) via file input or drag-drop; OR no file (default carrier used)
2	UI Layer	Canvas Bridge	File object → FileReader.readAsDataURL()
3	Canvas Bridge	UI Layer	Image loaded; width × height available for capacity calculation
4	User	UI Layer	String (secret message), String (optional key)
5	UI Layer	Key Scheduler	getShuffledIndices(channelCount, key: string)
6	Key Scheduler	LSB Engine	Array<number> — shuffled pixel channel indices
7	UI Layer	LSB Engine	encodeLSB(): ImageData pixels, message string, key string
8	LSB Engine	Canvas Bridge	Modified Uint8ClampedArray → ctx.putImageData()
9	Canvas Bridge	UI Layer	Result canvas ready; width, height, char count for metadata chips
10	User click	Canvas Bridge	canvas.toDataURL("image/png") → anchor download

3.5.2 Decode Flow

Step	From	To	Data passed
1	User	UI Layer	File object (encoded PNG); String (optional key)
2	UI Layer	Canvas Bridge	File → drawImage() → getImageData() on work canvas
3	Canvas Bridge	UI Layer	ImageData { width, height, data: Uint8ClampedArray }
4	UI Layer	Key Scheduler	getShuffledIndices(channelCount, key: string)
5	Key Scheduler	LSB Engine	Array<number> — same shuffled indices as encode step
6	UI Layer	LSB Engine	decodeLSB(): ImageData pixels, key string
7	LSB Engine	LSB Engine	Read 32 LSBs → reconstruct uint32 length N
8	LSB Engine	LSB Engine	Read N × 8 LSBs → Uint8Array(N)
9	LSB Engine	UI Layer	String (recovered message) via TextDecoder, or error if wrong key

3.5.2 Data Formats

Data Item	Format / Type	Notes
Carrier image (input)	File / Blob	PNG, JPEG, WebP, BMP, GIF — any format the browser Canvas can decode
Output stego image	PNG (lossless)	JPEG must never be used — lossy compression destroys LSBs
Default carrier	Canvas-generated PNG	512 × 512 gradient + noise; created on page load if no file uploaded
Secret message	UTF-8 string	Supports full Unicode; encoded to bytes via TextEncoder
Secret key	UTF-8 string (any length)	Hashed to a 32-bit seed via polynomial rolling hash
Pixel buffer	Uint8ClampedArray (RGBA)	Alpha byte (every 4th) always skipped; only R, G, B channels used
Bit index map	Array<number>	Shuffled channel indices; length = $W \times H \times 3$
Length header	uint32 big-endian (32 bits)	First 4 hidden bytes encode the message byte count; max ~2 GB
Capacity formula	$\text{floor}((W \times H \times 3 - 32) / 8)$	Max bytes available for the message payload after the header

3.6 Architecture-to-Technology Mapping

Component	Technology	Key APIs
UI Layer	<i>Vanilla JS (ES2020) + CSS3</i>	document.getElementById, addEventListener, DOM manipulation, CSS variables, @keyframes animations
Canvas Bridge	<i>HTML5 Canvas API + File API</i>	FileReader, HTMLCanvasElement, CanvasRenderingContext2D.drawImage / getImageData / putImageData, canvas.toDataURL("image/png")
LSB Engine	<i>Vanilla JavaScript</i>	Uint8ClampedArray, bitwise operators (& 0xFE, bit), TextEncoder, TextDecoder
Key Scheduler	<i>Vanilla JavaScript</i>	String.charCodeAt(), unsigned right shift (>>> 0), Math.floor() — no Web Crypto dependency
Capacity Estimator (inline)	<i>Vanilla JavaScript</i>	Simple arithmetic: $\text{floor}((W \times H \times 3 - 32) / 8)$, DOM innerText update
Styling / Layout	<i>Embedded CSS3</i>	CSS Grid, Flexbox, CSS custom properties (--accent, --bg, etc.), @keyframes, backdrop-filter
Typography	<i>Google Fonts (CDN)</i>	Syne (display/headings) + Space Mono (monospace body) via @import

Distribution	<i>Single HTML file</i>	No build tools, no npm, no transpilation — the .html file is the artefact
---------------------	-------------------------	---

3.7 Security Design Decisions

- No server, no data leakage: All processing happens in the browser. The cover image, the secret message, and the key never leave the device. This is the strongest possible privacy guarantee.
- Key-based pixel shuffling: Without the correct key, the bit sequence cannot be reconstructed — even if an attacker knows LSB steganography was used, the correct pixel reading order is unknown.
- Lossless PNG output: The stego image is always exported as PNG, preserving the exact pixel values. JPEG re-encoding would destroy the hidden data.
- Alpha channel untouched: Only R, G, B channels are used for embedding. The alpha channel is always preserved at its original value, preventing trivial detection via alpha channel analysis.
- 32-bit length header inside bitstream: The message length is encoded as part of the hidden payload, not derived from the key. An attacker cannot learn message length without extracting successfully.
- Wrong-key detection: TextDecoder is called with { fatal: true }. An incorrect key produces an invalid UTF-8 byte sequence, which throws an exception — the UI displays an error rather than garbage text.

Part 4 – Tools, Tech Stack & Development Phases

4.1 Complete Tools & Technology Reference

4.1.1 Core Development Technologies

Technology	Version / Type	Role in StegoCipher
HTML5	Standard	Application shell, semantic structure, file inputs, textarea, Canvas element for pixel processing, tab panel structure.
CSS3 (embedded)	Standard	Full dark theme via CSS variables (--bg, --accent, etc.), grid background pattern, noise texture overlay, CSS @keyframes animations (fadeIn, slideUp), responsive layout with Flexbox.
JavaScript ES2020 (embedded)	Vanilla — no framework	All application logic: UI layer, LSB engine, key scheduler, canvas bridge. Async/await with sleep(0) yields for non-blocking progress updates.
HTML5 Canvas API	Browser built-in	Primary pixel-access mechanism. drawImage() loads cover image; getImageData() extracts RGBA array; putImageData() writes modified pixels; toDataURL("image/png") exports the stego image.
HTML5 File API + FileReader	Browser built-in	Reads the user-uploaded image file into a Data URL for display and canvas loading. Also used for drag-and-drop via DataTransfer.files.
TextEncoder / TextDecoder	Browser built-in (Encoding API)	Converts the secret message string to UTF-8 bytes and back. TextDecoder called with { fatal: true } to detect wrong-key decoding failures.
Google Fonts CDN	External (load-time only)	Syne (display headings, weight 400–800) + Space Mono (monospace body). Loaded via CSS @import — no JS dependency.

4.1.2 Development Environment & Tooling

Tool	Purpose
Blob API / URL.createObjectURL	Used for the default carrier image generation only. Stego image download uses canvas.toDataURL() instead of canvas.toBlob().
Netlify hosting	Optional. The app works as a local HTML file. Netlify can be used for sharing a live URL.
Visual Studio Code + Live Server	Any editor works. Live Server is no longer mandatory since SubtleCrypto is not used.
GitHub	Version control and submission to teacher. Feature-branch workflow with pull requests.

4.2 Development Phases

Phase	Tasks & Deliverables	Exit Criteria
Phase 1 Foundation & Research	- Set up GitHub repository with correct directory structure and share with teacher. - Studied the HTML5 Canvas API — confirmed getImageData/putImageData provide full pixel-level read/write access. - Verified TextEncoder/TextDecoder support in all target browsers. - Defined the message binary format: [32-bit length header][message bits]. - Built default carrier image generator (canvas gradient + noise) for zero-friction testing.	Exit Criteria <i>GitHub repository live and shared. Default carrier loads on page open.</i>
Phase 2 Core LSB Engine	- Implement Key Scheduler — polynomial hash + LCG seed + Fisher-Yates shuffle. Write test: same key always produces same index array. - Implement LSB encoder — UTF-8 → bit array → write into pixel channel LSBs using shuffled index list. - Implement LSB decoder — read bit array from shuffled channels → reconstruct uint32 length → TextDecoder to string. - Wire capacity estimator into UI — update live bar on image load and textarea input. - Round-trip test in browser console: encode a 100-character message in a 512×512 PNG, decode it, assert exact match.	Exit Criteria <i>Round-trip test passes for a 100-character message. Wrong key returns an error, not garbage text.</i>
Phase 3 UI & Integration	- Design full HTML structure — two-tab layout (Encode / Decode), upload zones with drag-and-drop, secret message textarea, key field with visibility toggle, encode/decode action buttons, result canvas, metadata chips, download button. - Write embedded CSS — dark theme with CSS variables, animated grid background, noise texture overlay, progress bar animations, responsive layout for mobile. - Implement async progress feedback — sleep(0) yields every 20,000 bits to keep UI responsive; animated progress bar with percentage label. - Implement drag-and-drop image upload as an alternative to the file picker for both Encode and Decode panels. - End-to-end integration test — complete encode-then-decode round trip through full UI with three different cover images and messages.	Exit Criteria <i>Single stegocipher.html opens correctly from filesystem. Encode and decode complete without UI freezing. Downloaded PNG is visually identical to the cover image.</i>
Phase 4 Testing & Hardening	Optional Netlify deployment — verify HTTPS, add netlify.toml with security headers (X-Frame-Options, CSP).	Exit Criteria <i>Decoding gives back the same hidden message. App</i>

	- Usability testing with two external users — have them encode and decode without instructions. Fix any friction points in labels, error messages, or button placement. - Browser compatibility check — Chrome, Firefox, Edge, Safari. Confirm Canvas API, TextEncoder/TextDecoder, and Google Fonts load correctly in all targets.	<i>functional in all four target browsers.</i>
Phase 5 Documentation, Demo & Submission	- Write the GitHub documents — project overview, feature list, how to open locally (no server needed), how to use the application step-by-step with screenshots. - Prepare demonstration — three showcase scenarios: short text in a small image; long paragraph in a high-resolution photo; failed extraction with wrong key showing error message. - Final repository review — ensure all documents are in the repository, all code committed to main, and Netlify deployment (if used) reflects the final state.	Exit Criteria <i>GitHub repository complete. Demo scenarios tested and rehearsed. Group ready to present and answer questions.</i>

4.3 Project Timeline Summary

Phase	Key Deliverable	Status	Phase
Phase 1 — Foundation	Deliverables 1 & 2 (concepts + existing solutions) + default carrier generator	Complete	Phase 1 — Foundation
Phase 2 — LSB Engine	Core LSB encode/decode + key scheduler + capacity estimator	In Progress	Phase 2 — LSB Engine
Phase 3 — UI & Integration	Complete single-file HTML application (UI + integration)	Planned	Phase 3 — UI & Integration
Phase 4 — Testing	Browser testing + steganalysis + optional Netlify deployment	Planned	Phase 4 — Testing
Phase 5 — Documentation	Final docs, demo preparation, full submission	Planned	Phase 5 — Documentation

End of document